1 Lecture 12 (Oct 6) - Divide and Conquer: Multiplication; Backtracking: Text Segmentation

1.1 Fast Multiplication

Grade-school (lattice) multiplication of two n-digit numbers multiplies each digit of one number by the whole other number, then adds the results. Each digit costs $O(n)$, and there are n digits, so the total is $O(n^2)$. Can divide and conquer do better?

1.1.1 Cost model

Normally we use the RealRAM model, where arithmetic on any number costs $O(1)$. That defeats the point here. Instead, **count single-digit operations**: adding, subtracting or multiplying one pair of digits.

With this model, some operations on n-digit numbers are cheap:

- Adding or subtracting two n-digit numbers: $O(n)$ (school method)
- Multiplying by a power of 10 (padding with zeros): $O(n)$
- Splitting a number into its top and bottom digits: $O(n)$

Multiplication is the only hard part.

1.1.2 Split into halves

Let $m = \lceil \frac{n}{2} \rceil$. Split each input into its m most significant digits and its m least significant digits:

$$x = a \cdot 10^m + b \qquad y = c \cdot 10^m + d$$

Each of a, b, c, d has at most m digits. Multiply out (FOIL):

$$xy = ac \cdot 10^{2m} + (ad + bc) \cdot 10^m + bd$$

This reduces one n-digit product to **four** m-digit products.

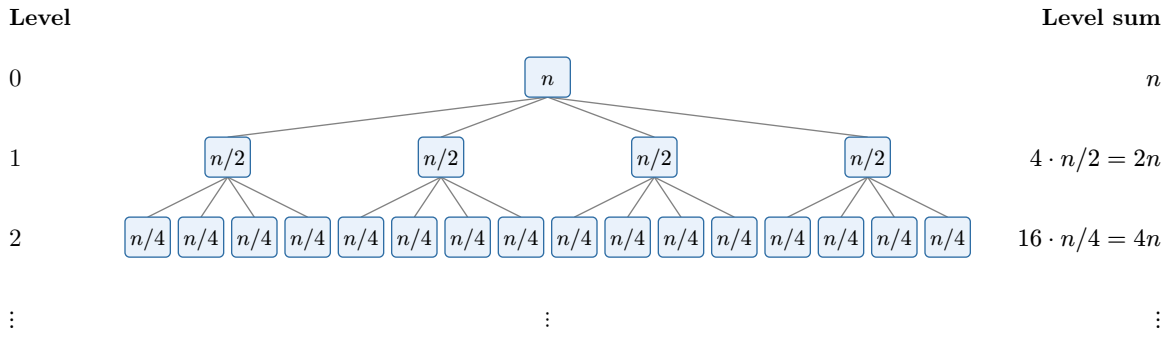
```
SplitMultiply(x, y, n):
  if n = 1
    return x · y                    ⟨⟨times table⟩⟩
  else
    m ← ⌈n/2⌉
    a ← ⌊x / 10^m⌋;  b ← x mod 10^m
    c ← ⌊y / 10^m⌋;  d ← y mod 10^m
    e ← SplitMultiply(a, c, m)
    f ← SplitMultiply(b, d, m)
    g ← SplitMultiply(b, c, m)
    h ← SplitMultiply(a, d, m)
    return 10^(2m)·e + 10^m·(g + h) + f
```

The `mod` and `/` by 10^m only chop the digit string, and the final line is padding and addition. All of that is $O(n)$.

1.1.3 Analysis

Let $T(n)$ be the number of single-digit operations. Four recursive calls of size $\frac{n}{2}$, plus $O(n)$ extra work:

$$T(n) = 4T\left(\frac{n}{2}\right) + O(n)$$



The level sums $n, 2n, 4n, \ldots$ form an **increasing geometric series** (ratio 2). A geometric series with ratio $\neq 1$ is proportional to its largest term. Here the largest term is the **bottom** level.

> **Key idea** **Increasing series, count the leaves:** Every leaf is a base case, so it does $O(1)$ work. The bottom level sum is $O(\#\text{ leaves})$.

- Each level has 4 times as many nodes as the one above.
- The size halves each level, so there are $\log_2 n$ levels.

$$\#\text{ leaves} = 4^{\log_2 n}$$

To simplify, use this log identity. You may swap the base and the argument of the log:

$$a^{\log_b c} = c^{\log_b a}$$

$$T(n) = O\left(4^{\log_2 n}\right) = O\left(n^{\log_2 4}\right) = \boxed{O(n^2)}$$

No improvement over grade school. Until 1960, people believed $O(n^2)$ was the best possible.

1.2 Karatsuba's Algorithm

Karatsuba (1960) noticed that the middle term needs only **one** new product. Add and subtract ac and bd:

$$ad + bc = ac + bd - (a - b)(c - d)$$

Check: $(a-b)(c-d) = ac - ad - bc + bd$, so $ac + bd - (a-b)(c-d) = ad + bc$.

The products ac and bd are already needed. So three recursive calls suffice: ac, bd and $(a-b)(c-d)$.

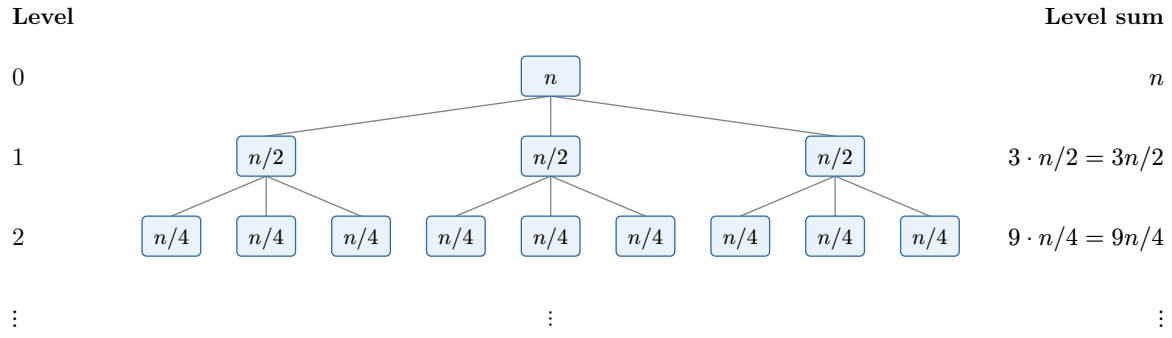
```
FastMultiply(x, y, n):
  if n = 1
    return x · y
  else
    m ← ⌈n/2⌉
    a ← ⌊x / 10^m⌋;  b ← x mod 10^m
    c ← ⌊y / 10^m⌋;  d ← y mod 10^m
    e ← FastMultiply(a, c, m)
    f ← FastMultiply(b, d, m)
    g ← FastMultiply(a − b, c − d, m)
    return 10^(2m)·e + 10^m·(e + f − g) + f
```

> **Watch out** $a-b$ and $c-d$ can be **negative**. Both still have at most m digits. Multiply their absolute values recursively, then fix the sign.

1.2.1 Analysis

Three recursive calls, still on size $\frac{n}{2}$, still $O(n)$ extra work:

$$T(n) = 3T\left(\frac{n}{2}\right) + O(n)$$



The level sums still increase geometrically (ratio $\frac{3}{2}$), so count the leaves. The depth is still $\log_2 n$, but each level now has 3 times as many nodes:

$$T(n) = O\left(3^{\log_2 n}\right) = O\left(n^{\log_2 3}\right) = \boxed{O(n^{1.585})}$$



> **Key idea** **Keep the number of recursive calls small:** The sizes shrink the same way in both algorithms. One call fewer turns n^2 into $n^{1.585}$, because the leaf count is $(\#\text{ calls})^{\text{depth}}$.

1.2.2 Later history

Chop the numbers into more pieces and combine the products more cleverly, and the time keeps dropping:

- 1971 (Schönhage and Strassen): $O(n \log n \log\log n)$
- 2019 (Harvey and van der Hoeven): $O(n \log n)$

The 2019 algorithm is purely theoretical. It only beats the 1971 one on numbers with more digits than there are particles in the universe.

1.2.3 Recursion tree shapes, all three seen

Level sums	Total	Example
Equal	level sum $\times$ # levels	Mergesort: $O(n \log n)$
Decreasing geometric	$O(\text{root})$	Mom select: $O(n)$
Increasing geometric	$O(\#\text{ leaves})$	Karatsuba: $O\left(n^{\log_2 3}\right)$

1.3 Backtracking

The next four lectures cover a second recursive strategy. Today is **only about correctness**. Thursday shows how to make these algorithms fast.

Many problems need a **sequence of decisions**:

- **List** every way to do a task
- Find the **best** way to do a task
- Decide whether the task is **possible** at all

> **Key idea** **Make one decision, try every option, recurse on the consequences:** Let the recursion fairy make all the other decisions.

How to use the recursive results depends on the goal:

Goal	Combine the recursive results by
List all solutions	Collect the results of every call
Find the best solution	Keep the best result
Is it possible?	Return true if **any** call returns true (OR)

1.4 Text Segmentation

Input: a string $A[1..n]$ with the spaces removed, and a black-box function IsWord that says whether a string is a word.

Output: true if A can be partitioned into substrings that are all words, false otherwise.

Ancient Latin was often written with no spaces, so this is a real problem. What counts as a "word" is the job of IsWord, not ours.

1.4.1 Decisions

Choose the words one at a time, left to right: what is the **next** word?

Example: `BLUESTEMUNITROBOTHEARTHANDSATURNSPIN`. Suppose the first four words are `BLUE STEM UNIT ROBOT`. The next word could be any of:

Next word	Left to segment
HE	ARTHANDSATURNSPIN
HEAR	THANDSATURNSPIN
HEART	HANDSATURNSPIN
HEARTH	ANDSATURNSPIN

We don't know which one is right, so **try all four**. For each, recursively ask whether the rest can be segmented. Any one success means the whole string works.

1.4.2 The recursive subproblem

What does the recursion fairy need to know? **Not** the words chosen so far. Only the **characters that remain** matter. Many different earlier choices can lead to the same remaining suffix, and the answer for that suffix is the same each time.

> **Key idea** Finding the right recursive subproblem is the main goal. Here it is: "can this **suffix** be segmented?" The algorithm follows almost automatically.

Base case: the empty string can always be segmented, into zero words. So return true.

```
Splittable(A[1..n]):
  if n = 0
    return True
  for i ← 1 to n
    if IsWord(A[1..i])
      if Splittable(A[i + 1..n])
        return True
  return False
```

Try every possible first word $A[1..i]$. If it is a word **and** the rest $A[i+1..n]$ can be segmented, succeed. If no first word works, fail.

Watch out **This is not greedy:** A greedy algorithm commits to one choice, for example "always take the shortest first word". That is probably wrong for some inputs. Backtracking tries **every** choice, so it is always correct.

1.4.3 Running time

Correct, but slow: it can make $O(2^n)$ calls to IsWord. Thursday shows how to make it fast.

1.4.4 Pass indices, not arrays

Copying the suffix into every recursive call is wasteful. Make $A[1..n]$ a **global** array. Every subproblem is a suffix, so one index identifies it: Splittable(i) asks whether $A[i..n]$ can be segmented.

```
Splittable(i):                          ⟨⟨A[1..n] is global⟩⟩
  if i > n
    return True
  for j ← i to n
    if IsWord(i, j)
      if Splittable(j + 1)
        return True
  return False
```

1.4.5 As a recurrence

The input is one integer and the output is a boolean. So Splittable is a mathematical function, and a recurrence can define it instead of pseudocode. Define:

- IsWord(i, j): true if and only if the substring $A[i..j]$ is a word
- Splittable(i): true if and only if the suffix $A[i..n]$ can be segmented

$$\text{Splittable}(i) = \begin{cases} \text{True} & \text{if } i > n \\ \bigvee_{j=i}^{n} (\text{IsWord}(i, j) \wedge \text{Splittable}(j+1)) & \text{otherwise} \end{cases}$$

The big $\vee$ tries every possible end j of the first word. **The answer to the original problem is Splittable(1).**

This recurrence describes the same algorithm as the pseudocode. Thursday turns this form into a fast algorithm almost mechanically.

1.5 Summary: the Backtracking Recipe

1. Think of the problem as a **sequence of decisions** (here: the next word).
2. Find the **recursive subproblem**: what the recursion fairy must know to finish (here: the remaining suffix, as one index i).
3. Try **every option** for one decision, and recurse on each.

4. Pick a **base case** where no decision is left (here: the empty suffix is true).
5. Write it as a **recurrence** on indices, and say which call answers the original problem.

> **Watch out** Don't skip steps 1 and 2, even after Thursday. The speedup only works once you have a correct recurrence to apply it to.

More backtracking examples are in the course notes and in Erickson's book, chapter 2.